

LLM Benchmark

June 29, 2026

Students:

Simone Migliaccio
Simone Spezia
Alberto Varini

Professors:

Allegra De Filippo
Michele Lombardi

Contents

LLM
Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

- 1 Introduction
- 2 Repository Structure
- 3 Problem Classification
- 4 Tools Used
- 5 Selected Models
- 6 Outputs and Evaluation
- 7 Results
- 8 Conclusions

Introduction

Goal and Keypoints

LLM
Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

Goal: build a reproducible benchmark for evaluating how LLMs solve PDDL planning problems and evaluate the reasoning capabilities.

Presentation structure

- Introduction
- Repository structure
- Problem classification
- Tools used
- Selected models
- Outputs and evaluation
- Results
- Conclusions

Repository Structure

General Structure

LLM Benchmark

Introduction

Repository Structure

Problem Classification

Tools Used

Selected Models

Outputs and Evaluation

Results

Conclusions

The repository is split into three main areas:

- **analysis:** notebooks, reports and generated figures.
- **archive:** reference papers, planning domains and PDDL support material.
- **Benchmark_Framework:** execution pipeline, tasks, model backends, validation and outputs.

LLM Benchmark/

```
├── analysis/
│   ├── domain_complexity/
│   ├── notebooks/
│   └── reports/
├── archive/
│   ├── Papers/
│   ├── Planning_Domains/
│   ├── References/
│   └── Quick_PDDL_Guide/
├── Benchmark_Framework/
│   └── ...
├── results/
└── README.md
```

General Structure

LLM Benchmark

`Benchmark_Framework` is the runnable benchmark layer. Only the core folders are shown here.

- *tasks*: selected PDDL domains and instances.
- *prompts* and *protocols*: prompt text, direct planning and repair settings.
- *models*: backend registries and adapter configuration.
- *runner*: suite and single-case execution logic.
- *evaluators*: parser, VAL validation, metrics and error taxonomy.
- *outputs*: raw, parsed and scored benchmark artifacts.
- *reporting*: post-run plots and JSON reports.
- *utils*: bundled VAL validator binaries.

`Benchmark_Framework/`

```
| evaluators/  
| models/  
| outputs/  
| prompts/  
| protocols/  
| reporting/  
| runner/  
| tasks/  
| utils/  
| run_benchmark.py
```

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

Benchmark Framework Overview

LLM
Benchmark

The framework is organized as a reproducible pipeline: the same task set can be tested with different prompts, protocols and model backends.

`tasks + prompts + protocols + model registries`



`run_benchmark.py`



`runner/run_suite.py → runner/run_case.py`



`model adapter → parser → VAL validator → metrics`



`outputs/raw + outputs/parsed + outputs/scored`

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

Why This Structure

LLM
Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

- **Comparable runs:** every model is evaluated on the same task, protocol and validation matrix.
- **Backend isolation:** model adapters keep HuggingFace, NVIDIA API, Ollama and llama.cpp behind the same generation interface.
- **External validation:** VAL checks the generated plans independently from the model text.
- **Traceable artifacts:** raw generations, parsed plans and scored validation results are stored separately.
- **Post-run analysis:** reports and plots are generated from saved JSON artifacts, not from manual inspection.

Problem Classification

Classification Purpose

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

Purpose: select a small but meaningful set of PDDL tasks from the available domains.

The classification has two complementary goals:

- assign a numerical difficulty score to each domain and instance;
- label each domain by its dominant reasoning type.

This makes the benchmark easier to interpret: model failures can be compared by both difficulty and planning style.

Implementation: Raw and Numerical Score

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

Difficulty scores are computed with
`Benchmark_Framework/scripts/score_domains_complexity.py`.

The score combines features that increase planning complexity:

- number of objects
- possible grounded actions
- number of goals
- numeric goal conditions
- numeric functions
- numeric preconditions
- numeric effects
- metric optimization
- topology structure

Each instance receives a raw score, then scores are normalized within the domain and summarized at domain level.

Implementation: Raw and Numerical Score II

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

The raw score is a weighted logarithmic combination, so no single count dominates the result:

$$RawScore = 0.45 \cdot \log(1 + Nactions) + 0.20 \cdot \log(1 + Ngoals) + 0.20 \cdot \log(1 + NumScore) + 0.15 \cdot Topology$$

$$NumScore = 1.0 \cdot NumEffects + 2.0 \cdot NumPreconditions + 3.0 \cdot NumGoalConditions + 1.0 \cdot HasMetric$$

The example shows the *counters* domain summary: instance scores plus aggregate raw-score statistics.

```
{  
  "domain": "counters",  
  "instances": 20,  
  "total_raw_score": 59.765337,  
  "min_raw_score": 1.820528,  
  "median_raw_score": 3.177554,  
  "mean_raw_score": 2.988267,  
  "max_raw_score": 3.679334,  
  "domain_difficulty_0_100": 18.130829  
},
```

Implementation: Reasoning Type

LLM Benchmark

Introduction

Repository Structure

Problem Classification

Tools Used

Selected Models

Outputs and Evaluation

Results

Conclusions

Difficulty alone is not enough: two domains can be equally hard while requiring different reasoning skills.

We therefore assign each domain a dominant reasoning category:

- numerical optimization
- symbolic logic
- temporal planning
- logistics
- spatial planning

Mixed domains are labeled by the reasoning type that is most central to solving the problem.

Problems Chosen

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

We selected six domains from the three reasoning categories with enough coverage: numerical optimization, logistics and spatial planning.

Hard Set

- settlersnumeric (numerical optimization)
- rover (logistics)
- block-grouping (spatial)

Easy Set

- fo-counters (numerical optimization)
- expedition (logistics)
- fo-sailing (spatial)

Temporal planning and symbolic logic were excluded from the final set because only one domain was available for each category.

Tools Used

Tools Used

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

The benchmark relies on a small set of tools, each with a specific role:

- **Leonardo HPC by CINECA:** main compute platform for local model execution.
- **Hugging Face:** source for local inference setup and download of open-source models.
- **whichLLM:** model-selection support based on available hardware on Leonardo.
- **VAL validator:** external PDDL plan validator.
- **NVIDIA API:** alternative backend to use open-source models without a local powerful machine.

Both Leonardo and the NVIDIA API were used: Leonardo for controlled local execution with direct GPU access, and NVIDIA API as a hosted backend for additional model runs.

Selected Models

Selected Models

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

Model selection was guided by two constraints:

- compatibility with the available Leonardo hardware;
- coverage of recent open LLM families available through Hugging Face.

The local evaluation shown in the results includes:

- Gemma 4 31B IT
- Nemotron 3 Nano 30B A3B
- Phi-4
- Qwen 3 6/27B

Recommended Models					
#	Model	Quant	Fit / VRAM	Speed	Published
1	deepseek-ai/DeepSeek-V4-Flash 280.0B (13.0Ba)	Q6_K	Full GPU 217.6 GB	13.5 tok/s ~	2026-04-22
2	XiaoMiMo/MiMo-V2.5 210.0B (19.0Ba)	Q6_K	Full GPU 238.1 GB	12.4 tok/s ~	2026-04-27
3	Qwen/Qwen3.6-27B 27.8B	Q6_K	Full GPU 24.4 GB	34.6 tok/s ~	2026-04-21
4	Qwen/Qwen3-235B-A22B 235.1B (22.0Ba)	Q6_K	Full GPU 181.7 GB	16.3 tok/s ~	2025-04-27
5	google/gemma-4-31B-it 32.7B	Q6_K	Full GPU 28.6 GB	29.4 tok/s ~	2026-03-11
6	Qwen/Qwen3-Next-80B-A3B-Instruct 81.3B (3.0Ba)	Q8_0	Full GPU 81.8 GB	32.5 tok/s ~	2025-09-09
7	openai/gpt-oss-120b 120.4B (5.16a)	Q6_K	Full GPU 92.8 GB	31.9 tok/s ~	2025-08-04
8	MiniMaxAI/MiniMax-M2.5 226.7B (10.8Ba)	Q6_K	Full GPU 175.3 GB	16.8 tok/s ~	2026-02-12
9	Qwen/Qwen3-30B-A3B 30.5B (3.0Ba)	Q8_0	Full GPU 31.6 GB	86.6 tok/s ~	2025-04-27
10	mistralai/Mistral-Large-Instruct-2411 122.6B	Q5_K_M	Full GPU 98.3 GB	9.6 tok/s ~	2024-11-14

Outputs and Evaluation

Output Structure

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

Each benchmark case produces three aligned JSON artifacts:

- **Raw:** prompt messages, model output and provider-side reasoning text when available.
- **Parsed:** extracted PDDL actions, final plan text and formatting issues.
- **Scored:** VAL validation result, solved flag, iterations used and repair feedback.

The suite also writes `suite_result.json`, which summarizes the run matrix and is the entry point for post-run analysis.

This split keeps generation, extraction and evaluation independent, so parser errors and planning failures can be diagnosed separately.

Evaluation Metrics Scope

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

The metrics are designed to answer three different questions:

- **Did the model solve the planning task?**
- **Was the produced action sequence grounded in the PDDL domain?**
- **Did retries improve the answer or only resample attempts?**

For this reason, the evaluation does not rely only on final success. It combines success, executability, hallucination, precondition awareness and retry behavior.

Core Metrics

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

- **SR**: fraction of instances solved by the model.
- **FASR**: first-attempt success rate; measures zero-shot planning ability.
- **IWSR**: iteration-weighted success rate; rewards valid plans found with fewer retries.
- **Retry Gap**: $SR - FASR$; high values mean the model depends on repair attempts.
- **Executability Ratio**: fraction of the plan executable before the first precondition failure.
- **Hallucination Rate**: fraction of actions or objects not grounded in the PDDL files.
- **PAS**: precondition awareness score; separates sequencing errors from fabricated states.
- **CoTaL**: alignment between reasoning actions and the final extracted plan when reasoning is available.

Cross-Domain and Cross-Model Analysis

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

- **Within-domain ranking:** compares models inside each domain using normalized metric values.
- **Cross-domain consistency:** checks whether a model is robust across different reasoning categories.
- **Failure mode taxonomy:** combines hallucination, executability and PAS to identify why plans fail.
- **Difficulty analysis:** compares easy, medium and hard instances to see where performance degrades.

Composite Planning Score

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

The Composite Planning Score summarizes complementary planning signals in one value:

$$PS = 0.25 \cdot FASR + 0.20 \cdot IWSR + 0.20 \cdot ER + 0.20 \cdot (1 - Hallucination) + 0.15 \cdot PAS$$

- **FASR** has the highest weight because it cannot be inflated by retries.
- **IWSR** and **ER** capture repair efficiency and structural plan quality.
- **Grounding** is represented by inverse hallucination rate.
- **PAS** adds information about local precondition reasoning.

Results

Evaluation Snapshot

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

The plots shown here come from the generated report `results/LLM_classification_test`.

- **Rows:** 267 evaluated task instances.
- **Models:** Gemma, Nemotron, Phi-4 and Qwen.
- **Domains:** block-grouping, expedition, fo-counters, fo-sailing, rover and settlersnumeric.
- **Protocol:** iterative repair.
- **Difficulty tiers:** easy, medium and hard.

Success and Hallucination

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

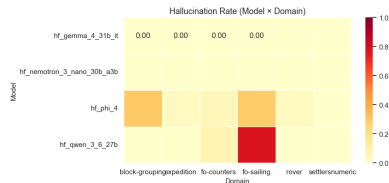
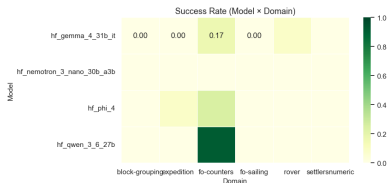
Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

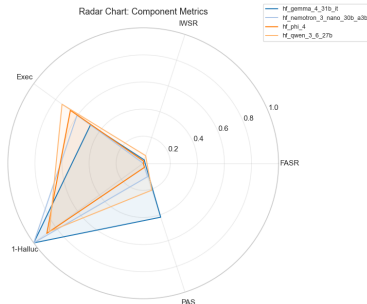
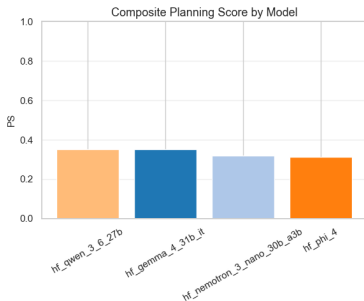


Final success and grounding errors must be read together: a model can use legal-looking actions while still failing the global goal.

Composite Score and Capability Profile

LLM Benchmark

Introduction
Repository
Structure
Problem
Classification
Tools Used
Selected
Models
Outputs and
Evaluation
Results
Conclusions



The composite score gives a compact ranking; the radar chart shows which capabilities drive each score.

Conclusions

Conclusions

LLM Benchmark

Introduction

Repository
Structure

Problem
Classification

Tools Used

Selected
Models

Outputs and
Evaluation

Results

Conclusions

- The benchmark provides a reproducible way to compare LLMs on planning tasks, using the same domains, protocols and validator.
- VAL validation keeps final scoring independent from model wording and parser interpretation.
- The raw / parsed / scored split makes failures easier to inspect and compare.
- The metrics separate final success, grounding, executability, retry dependence and reasoning alignment.
- Future work can extend the task set, add more temporal and logic domains, and compare additional model backends.